
[PFE - Condition Monitoring Research]

Bearing Signal Simulator

Vibration Analysis &
Fault Diagnosis Platform

A professional-grade web application for simulating
rolling-element bearing vibration signals,
performing real-time spectral analysis, and
diagnosing mechanical faults.

Python 3.11+

FastAPI

NumPy / SciPy

ECharts 5

WebSocket

Based on Harris Bearing Theory

Contents

1	Project Overview	3
2	Theoretical Background	3
2.1	Vibration-Based Bearing Diagnosis	3
2.2	Bearing Characteristic Frequencies	3
2.3	Default Bearing: SKF 6205 (CWRU Dataset)	4
2.4	Impulse Response Model (SDOF)	4
2.5	Envelope Analysis (Hilbert Transform)	5
3	Architecture & Technology Stack	5
3.1	System Architecture	5
3.2	Technology Stack	6
4	Project Structure	6
4.1	Module Descriptions	6
4.1.1	core/physics.py	6
4.1.2	core/generator.py	6
4.1.3	core/features.py	7
4.1.4	core/simulation_engine.py	7
5	Installation & Setup	7
5.1	Prerequisites	7
5.2	Steps	7
6	How to Use the Application	7
6.1	One-Shot Simulation (Static Mode)	8
6.2	Real-Time Streaming Mode	8
6.3	Fault Injection	8
7	Parameter Reference	8
7.1	Bearing Geometry	9
7.2	Kinematics & Signal	9
7.3	Fault Configuration	9
7.3.1	Severity Model	10
8	Understanding the Charts	10
8.1	Time Domain (Oscilloscope)	10
8.2	Power Spectral Density (Welch PSD)	10
8.3	Envelope Spectrum (Hilbert Transform)	10
9	Health Diagnostics System	11
9.1	Traffic Light Status	11
9.2	Diagnostic Thresholds	11
9.3	Statistical Indicators	11
10	Signal Generation Pipeline	12
11	API Reference	12

- 11.1 REST Endpoints 12
 - 11.1.1 GET / 12
 - 11.1.2 GET /api/params 12
 - 11.1.3 POST /api/simulate 13
 - 11.2 WebSocket Endpoint 13
 - 11.2.1 WS /ws/stream 13
- 12 Deployment 14
 - 12.1 Local Development 14
 - 12.2 Production (Render.com) 14
- 13 Frequently Asked Questions 14
- References 15

1 Project Overview

⊗ **Summary:** This platform simulates rolling-element bearing vibration signals under healthy and faulty conditions, providing real-time spectral analysis and automated health diagnostics through a web-based dashboard.

The **Bearing Signal Simulator** enables engineers and researchers to:

- **Configure bearing geometry** — number of balls, diameters, and contact angle
- **Simulate 5 operating states** — Healthy, Outer Race fault, Inner Race fault, Ball fault, and Cage fault
- **Control fault severity** from 0% (barely detectable) to 100% (catastrophic)
- **Visualize signals** in time-domain, frequency-domain (PSD), and envelope spectrum
- **Stream data in real-time** via WebSocket with live-updating charts
- **Receive automated health diagnostics** with traffic-light indicators and maintenance recommendations

The physics model is grounded in **Harris Bearing Theory** and the **McFadden & Smith single-point defect model**.

2 Theoretical Background

2.1 Vibration-Based Bearing Diagnosis

When a bearing has a defect (crack, spall, pit) on one of its components, every time a rolling element passes over that defect, it creates a mechanical **impact**. This impact excites the bearing housing structure, causing it to vibrate at its natural frequency. By analyzing the *repetition rate* of these impacts, we can determine exactly which bearing component is damaged.

2.2 Bearing Characteristic Frequencies

Every bearing has four **Characteristic Defect Frequencies** strictly determined by its geometry and shaft speed:

Frequency	Full Name	Formula
BPFO	Ball Pass Freq. Outer Race	$BPFO = \frac{N_b}{2} f_r \left(1 - \frac{d}{D} \cos \varphi \right)$
BPMI	Ball Pass Freq. Inner Race	$BPMI = \frac{N_b}{2} f_r \left(1 + \frac{d}{D} \cos \varphi \right)$
BSF	Ball Spin Frequency	$BSF = \frac{D}{2d} f_r \left[1 - \left(\frac{d}{D} \cos \varphi \right)^2 \right]$
FTF	Fundamental Train Freq.	$FTF = \frac{1}{2} f_r \left(1 - \frac{d}{D} \cos \varphi \right)$

Table 1: Bearing characteristic defect frequencies (Harris Theory)

where: N_b = number of rolling elements, f_r = shaft rotation frequency (Hz), d = ball diameter, D = pitch diameter, φ = contact angle.

★ **Key Insight:** If a defect exists on the outer race, impacts repeat at exactly BPFO Hz. If it is on the inner race, they repeat at BPMI Hz. This is the mathematical foundation of bearing fault diagnosis.

2.3 Default Bearing: SKF 6205 (CWRU Dataset)

Parameter	Symbol	Value
Number of Balls	N_b	8
Ball Diameter	d	8 mm
Pitch Diameter	D	40 mm
Contact Angle	φ	0°
Shaft Speed	f_r	30 Hz (1800 RPM)
Computed BPFO	—	96 Hz
Computed BPMI	—	144 Hz
Computed BSF	—	72 Hz
Computed FTF	—	12 Hz

Table 2: Default bearing parameters — SKF 6205-2RS

2.4 Impulse Response Model (SDOF)

When a ball hits a defect, the bearing housing structure vibrates. This is modeled as a **Single-Degree-of-Freedom (SDOF) damped oscillator**:

$$h(t) = e^{-\zeta \omega_n t} \cdot \sin(\omega_d t)$$

- $\omega_n = 2\pi f_n$ — natural angular frequency

- $\omega_d = \omega_n \sqrt{1 - \zeta^2}$ — damped natural frequency
- f_n — natural frequency of the structure (default: 3000 Hz)
- ζ — damping ratio (default: 0.02)

2.5 Envelope Analysis (Hilbert Transform)

The raw vibration signal contains two components: high-frequency structural ringing at ≈ 3000 Hz, and low-frequency impact repetition (the fault frequency). Envelope analysis separates them via a three-step process:

1. Apply the **Hilbert Transform** to obtain the analytic signal
2. Take the absolute value to extract the **amplitude envelope**
3. Compute the **FFT of the envelope** to reveal the envelope spectrum

The envelope spectrum produces peaks precisely at the characteristic defect frequencies, enabling exact fault identification.

3 Architecture & Technology Stack

3.1 System Architecture

□ Client-Server Architecture

```

+-----+
|      BROWSER  (Client)      |
| [Params Panel]  [ECharts Time] |
| [ECharts PSD]   [Health Monitor] |
|      REST (POST) / WebSocket  |
+-----+-----+
|                               |
+-----v-----+
|      FastAPI Server (Python)  |
|                               |
| POST /api/simulate             |
| WS   /ws/stream               |
|                               |
| core/                         |
|   physics.py    -> frequencies |
|   generator.py  -> signals     |
|   features.py   -> PSD/envelope |
|   simulation_engine.py         |
+-----+

```

3.2 Technology Stack

Layer	Technology	Purpose
Backend	Python 3.11+	Core simulation logic
Web Framework	FastAPI	REST API + WebSocket server
Math Libraries	NumPy, SciPy, Pandas	Signal processing & FFT
Frontend	HTML / CSS / JS	Dashboard UI
Charting	ECharts 5 (CDN)	High-performance chart rendering
Typography	Inter, JetBrains Mono	Professional typography

Table 3: Full technology stack

4 Project Structure

▣ Directory Layout

```
Simulation_Signaux_Roulement/  
  core/  
    __init__.py  
    physics.py          # Harris frequencies + impulse response  
    generator.py        # Signal generation engine  
    features.py         # Feature extraction (RMS, PSD, envelope)  
    simulation_engine.py# BearingSimulator orchestrator  
  static/  
    index.html          # Dashboard layout  
    style.css           # Design system (dark theme)  
    app.js              # Chart rendering + API integration  
  app.py                # FastAPI server entry point  
  requirements.txt      # Python dependencies  
  README.md            # Project documentation
```

4.1 Module Descriptions

4.1.1 core/physics.py

- `bearing_freqs(fr, Nb, d, D, phi)` — computes BPFO, BPFI, BSF, FTF
- `impulse_response(t, fn, zeta)` — SDOF damped oscillator $h(t)$
- `DEFAULT_PARAMS` — dictionary of default SKF 6205 parameters

4.1.2 core/generator.py

- `generate_healthy_signal(t, fr, snr_db)` — Gaussian noise + rotational harmonics
- `generate_defect_signal(t, f_def, h, ...)` — impact train convolution + AM/FM modulation

- `generate_signal(signal_type, params)` — factory function routing to correct generator

4.1.3 core/features.py

- `compute_time_stats(sig)` — RMS, kurtosis, skewness, crest factor, peak-to-peak
- `compute_psd(sig, fs)` — Welch Power Spectral Density
- `compute_envelope_spectrum(sig, fs)` — Hilbert envelope spectrum
- `compute_all_features(sig, fs)` — aggregates all features

4.1.4 core/simulation_engine.py

- `BearingSimulator.simulate()` — one-shot full simulation with all features
- `BearingSimulator.stream_chunk()` — generate one streaming chunk
- `BearingSimulator.reset_stream()` — reset streaming state

5 Installation & Setup

5.1 Prerequisites

- Python 3.10 or newer
- pip package manager
- A modern web browser (Chrome, Firefox, Edge)

5.2 Steps

» Quick Start

```
# 1. Clone the repository
git clone https://github.com/fouad3966/Bearing_Simulation_PFE.git
cd Bearing_Simulation_PFE

# 2. Install Python dependencies
pip install -r requirements.txt

# 3. Start the server
python -m uvicorn app:app --reload --port 8000

# 4. Open in your browser
# Navigate to: http://localhost:8000
```

6 How to Use the Application

6.1 One-Shot Simulation (Static Mode)

1. **Set bearing parameters** in the left panel (or leave defaults for SKF 6205)
2. **Select a fault type** — Healthy, Outer Race, Inner Race, Ball, or Cage
3. **Adjust severity** using the slider (0% = barely detectable, 100% = catastrophic)
4. **Adjust SNR** — higher = cleaner signal, lower = more noise
5. **Click "Simulate"** — all three charts update simultaneously
6. The **Health Monitor** panel shows diagnostic indicators

▲ **Note:** Clicking "Simulate" multiple times will produce slightly different graphs due to random noise, phase, and jitter. This is intentional — it mirrors real-world sensor behavior. The **underlying diagnostic features** (envelope spectrum peaks) remain consistent.

6.2 Real-Time Streaming Mode

1. Configure your parameters (fault type, severity, etc.)
2. Click the green **"Stream"** button
3. All three charts update live — Time Domain at ≈ 12 fps, PSD and Envelope Spectrum every ≈ 0.5 s
4. The Health Monitor updates continuously
5. The status badge shows *Streaming* with an elapsed-time counter

6.3 Fault Injection

◆ **The "Wow" Feature:** Simulate a sudden bearing failure in real time.

1. Start streaming in **Healthy** mode
2. While streaming, select a fault type (e.g., **Outer Race**)
3. Set the desired severity
4. Click the orange **"Inject Fault"** button
5. Watch all charts instantly transition from healthy to faulty
6. The traffic light changes from **green** to **yellow** or **red**

7 Parameter Reference

7.1 Bearing Geometry

Parameter	Symbol	Default	Unit	Description
Number of Balls	N_b	8	—	Rolling elements inside the bearing
Ball Diameter	d	8	mm	Diameter of each ball
Pitch Diameter	D	40	mm	Distance between centers of opposing balls
Contact Angle	φ	0	deg	Angle between load line and radial plane

Table 4: Bearing geometry parameters

7.2 Kinematics & Signal

Parameter	Symbol	Default	Unit	Description
Shaft Speed	f_r	30	Hz	Shaft rotation frequency (30 Hz = 1800 RPM)
Sample Rate	f_s	12000	Hz	Data points per second (Nyquist limit = $f_s/2$)
Nat. Frequency	f_n	3000	Hz	Resonant frequency of the bearing housing
Damping Ratio	ζ	0.02	—	How quickly structural ringing decays

Table 5: Kinematics and structural parameters

7.3 Fault Configuration

Parameter	Default	Range	Description
Fault Type	Healthy	5 options	Which bearing component has the defect
Severity	50%	0–100%	How severe the damage is
SNR	20 dB	–10 to 60 dB	Signal-to-noise ratio
Duration	2 s	0.1–10 s	Length of the simulated signal (one-shot)

Table 6: Fault configuration parameters

7.3.1 Severity Model

Effect	Low Severity (10%)	High Severity (90%)
Impact Amplitude	Very weak	Very strong
Timing Jitter	High (irregular impacts)	Low (regular impacts)
SNR Reduction	Minimal	Significant
Non-linearity	None	Plastic deformation distortion
FM Modulation	None	Present (speed fluctuations)

Table 7: Effect of severity level on signal characteristics

8 Understanding the Charts

8.1 Time Domain (Oscilloscope)

What it shows: Raw vibration signal as recorded by an accelerometer over time.

Axes: X = time (s) — Y = vibration amplitude (acceleration, a.u.)

Healthy signal: Random noise with no visible pattern.

Faulty signal: Periodic bursts at regular intervals — spacing equals $1/f_{\text{defect}}$.

Example: BPFO = 96 Hz $\Rightarrow$ bursts every ≈ 10.4 ms.

8.2 Power Spectral Density (Welch PSD)

What it shows: Vibration energy distribution across frequencies, computed via Welch’s averaged periodogram method.

Axes: X = frequency (Hz, 0 to $f_s/2$) — Y = power spectral density.

Key feature: A dominant peak near $f_n = 3000$ Hz (structural resonance). With faults, this peak grows significantly. The PSD alone *cannot* identify which component is faulty.

8.3 Envelope Spectrum (Hilbert Transform)

What it shows: The modulation pattern of the signal — revealing the repetition rate of impacts.

Axes: X = frequency (0–600 Hz) — Y = envelope magnitude.

Colored markers:

- **Yellow** — BPFO (Outer Race)
- **Red** — BPFI (Inner Race)
- **Purple** — BSF (Ball)
- **Cyan** — FTF (Cage)

A spectral peak aligned with a marker confirms a defect on that component.

9 Health Diagnostics System

9.1 Traffic Light Status

Indicator	Status	Meaning
●	NORMAL	All indicators within safe limits
●	WARNING	Early-stage defect suspected
●	DANGER	Significant defect — immediate action required

Table 8: Health traffic-light system

9.2 Diagnostic Thresholds

Indicator	Normal (Green)	Warning (Yellow)	Danger (Red)
Kurtosis	< 4	$4 - 10$	> 10
RMS	< 0.1	$0.1 - 0.3$	> 0.3
Crest Factor	< 4	$4 - 6$	> 6

Table 9: Diagnostic threshold levels

9.3 Statistical Indicators

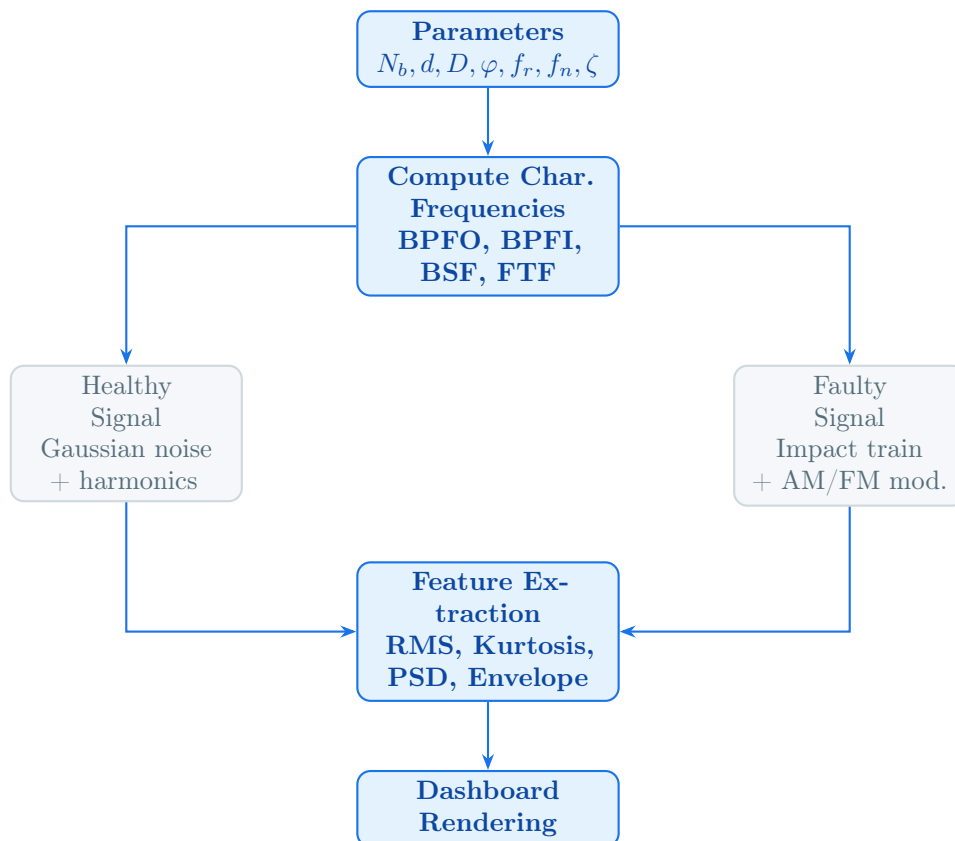
Indicator	What It Measures	Why It Matters
RMS	Root mean square — overall vibration energy	Increases with fault severity
Kurtosis	"Peakedness" of the signal distribution	Healthy ≈ 3 ; impacts push it $\gg 3$
Crest Factor	Ratio of peak value to RMS	Detects sharp transient impacts
Skewness	Asymmetry of signal distribution	Indicates directional fault effects

Table 10: Statistical health indicators explained

10 Signal Generation Pipeline

⊗ Pipeline Overview

The simulator routes through a clearly defined computation pipeline, from raw parameters to dashboard output.



🕒 **Why results change each run:** Even with identical parameters, each simulation produces a slightly different signal. Random Gaussian noise, impact timing jitter, and random harmonic phase are intentional — they mirror real-world sensor behavior. The diagnostic features (envelope peaks, characteristic frequencies) remain **consistent**.

11 API Reference

11.1 REST Endpoints

11.1.1 GET /

Serves the main dashboard HTML page.

11.1.2 GET /api/params

Returns default parameters and their UI metadata.

→ Response

```
{
  "defaults": {"fs": 12000, "T": 2.0, "Nb": 8, ...},
  "descriptions": {
    "fs": {"label": "Sampling Frequency",
           "unit": "Hz", "min": 1000, "max": 100000}
  }
}
```

11.1.3 POST /api/simulate

Run a one-shot simulation.

← Request Body

```
{
  "signal_type": "OuterRace",
  "severity":    0.7,
  "duration":    2.0,
  "fs":          12000,
  "Nb":          8,
  "d":           0.008,
  "D":           0.04,
  "phi":         0.0,
  "fr":          30.0,
  "fn":          3000.0,
  "zeta":        0.02,
  "snr_db":      20.0
}
```

Response: JSON with `time[]`, `signal[]`, `features{}`, `char_freqs{}`, `params{}`.

11.2 WebSocket Endpoint

11.2.1 WS /ws/stream

← Client → Server Messages

```
{"action": "start", "params": {...}}
{"action": "stop"}
{"action": "inject_fault", "type": "InnerRace", "severity": 0.8}
{"action": "update_params", "params": {...}}
```

→ Server → Client Message (every 80ms)

```
{
  "time":    [...],
  "signal":  [...],
  "stats": {
    "rms": 0.18, "kurtosis": 7.4,
    "crest_factor": 5.2, "skewness": 0.1
  }
}
```

```
},
"char_freqs": {
  "BPFO": 96.0, "BPFI": 144.0,
  "BSF": 72.0, "FTF": 12.0
},
"psd": {"freqs": [...], "values": [...]},
"envelope": {"freqs": [...], "values": [...]}
}
```

⊗ **Performance note:** The `psd` and `envelope` fields are included every 5th chunk (≈ 0.5 s intervals) to balance performance with real-time spectral updates.

12 Deployment

12.1 Local Development

```
pip install -r requirements.txt
python -m uvicorn app:app --reload --port 8000
```

12.2 Production (Render.com)

This project is configured for deployment on [Render.com](#). The server entry point is `app:app` served with `uvicorn`.

13 Frequently Asked Questions

Q Why does the signal look different every time I click "Simulate"?

The simulation uses stochastic processes — Gaussian noise, impact jitter, and random phase — to model real-world sensor behavior. The underlying diagnostic features (envelope peaks, characteristic frequencies) remain consistent.

Q What do the dashed colored lines on the Envelope Spectrum mean?

They mark the characteristic defect frequencies: **Yellow** = BPFO, **Red** = BPFI, **Purple** = BSF, **Cyan** = FTF.

If a spectral peak aligns with one of these lines, it confirms a defect on that bearing component.

Q What severity should I use for a realistic demo?

20–30%: Very early-stage fault — hard to see in time domain, detectable in envelope spectrum.

50–60%: Moderate fault — visible impacts, clear envelope peaks.

80–100%: Severe fault — dramatic impacts, strong diagnostic indicators.

Q Why is Kurtosis ≈ 3 for healthy signals?

A Gaussian (normal) distribution has standard kurtosis of 3. Healthy bearing vibration is approximately Gaussian noise, so kurtosis ≈ 3 . Impacts from defects create sharp peaks that push kurtosis well above 3.

References

1. Harris, T.A. (2001). *Rolling Bearing Analysis*, 4th Edition. Wiley.
2. McFadden, P.D. & Smith, J.D. (1984). *Model for the vibration produced by a single point defect in a rolling element bearing*. Journal of Sound and Vibration.
3. Case Western Reserve University Bearing Data Center. <https://engineering.case.edu/bearingdatacenter>

Developed as part of a PFE (*Projet de Fin d'Études*)
for vibration-based bearing condition monitoring research.